

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

Ордена Трудового Красного Знамени

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«Московский технический университет связи и информатики»

Кафедра «Математическая кибернетика и информационные технологии»

Отчет по лабораторной работе №5

по дисциплине «Информационные технологии и программирование»

Выполнил: студент группы

БПИ2401

Старков Дмитрий Константинович

Проверил:

Харрасов Камиль Раисович

Москва, 2025

Содержание

Цель работы	2
Ход работы.....	2
Вывод.....	6

Цель работы

Освоить строки и регулярные выражения в языке Java.

Ход работы

Начнем с выполнения первого задания:

```
1  import java.util.regex.*;
2
3  public class NumberFinder {
4
5      public static void main(String[] args) {
6          Pattern pattern = null;
7          try {
8              String text = "The price of the product is $19.99 and the discount is 20%, and my iq is 99";
9
10             // Pattern pattern = Pattern.compile("(");
11             pattern = Pattern.compile("\\d+(\\.\\d+)?");
12             Matcher matcher = pattern.matcher(text);
13
14             boolean found = false;
15
16             while (matcher.find()) {
17                 System.out.println(matcher.group());
18                 found = true;
19             }
20
21             if (!found) {
22                 System.out.println("No numbers found");
23             }
24
25             } catch (PatternSyntaxException e) {
26                 System.out.println("Error in syntax: " + e.getDescription());
27             } catch (Exception e) {
28                 System.out.println("Error: " + e.getMessage());
29             }
30     }
31 }
32
```

Первой строкой в методе main я инициализирую паттерн как null, чтобы в дальнейшем проверить корректность его переопределения в блоке try-catch. Далее сам блок try-catch, в котором я задаю искомую строку, потом идет закомментированная строка с присвоением паттерну невалидного значения я вызова ошибки, а уже после нормальное присвоение. Сам паттерн выглядит вот так `\\d+(\\.\\d+)?` что означает `\\d` – любая цифра, + - один или более раз (не использовал *, так как нам обязательно нужна хотя бы одна цифра), затем в скобках `\\.\\d+` - точка и любая цифра 1 или более раз. После скобок стоит ?, который делает дробную часть опциональной. Ну и дальше простая проверка совпадений и вывод результата в консоль, а в конце обработка исключений.

Теперь второе задание

```
1  import java.util.regex.*;
2  import java.util.Scanner;
3
4  public class PasswordValidator {
5
6      public static void main(String[] args) {
7          try {
8              Scanner scanner = new Scanner(System.in);
9              System.out.print("Enter password: ");
10             String password = scanner.nextLine();
11
12             Pattern pattern = Pattern.compile("(?=.*[A-Z])(?=.*\\d)[A-Za-z0-9]{8,16}$");
13
14             Matcher matcher = pattern.matcher(password);
15
16             if (matcher.matches()) {
17                 System.out.println("Password is valid.");
18             } else {
19                 System.out.println("Password is not valid.");
20             }
21
22         } catch (PatternSyntaxException e) {
23             System.out.println("Regex error: " + e.getDescription());
24         } catch (Exception e) {
25             System.out.println("Unexpected error: " + e.getMessage());
26         }
27     }
28 }
29
```

В этом задании я создал паттерн для проверки всех необходимых условий к содержанию введенного пароля. Разберем паттерн подробнее. Само выражение стоит между `^` и `$`, что означает просмотр всей строки. Далее в скобках идет так называемый lookahead оператор `?=.*` который «заглядывает» вперед и проверяет условие, а именно `.*[A-Z]` – любой символ (.) сколько угодно раз или 0 раз (*) и любая из заглавных букв латинского алфавита ([A-Z]). Иными словами мы ищем строку любой длины, после которой идет заглавная буква. Второе условие в скобках аналогичное, но для проверки на наличие хотя бы одной цифры. Как только эти условия были проверены, начинается проверка алфавита пароля – только буквы латинского алфавита и цифры, причем длина от 8 до 16 символов (`{8, 16}`).

В третьем задании необходимо было найти комбинации заглавной буквы после строчной.

```
1  import java.util.regex.*;
2
3  public class CapitalAfterLower {
4
5      public static void main(String[] args) {
6          String text = "aB cD Ef gH iJ Kl mN oP qR sT Uv wX yZ";
7
8          Pattern pattern = Pattern.compile("[a-z][A-Z]");
9          Matcher matcher = pattern.matcher(text);
10
11         StringBuffer result = new StringBuffer();
12
13         while (matcher.find()) {
14             String match = matcher.group();
15
16             matcher.appendReplacement(result, "!" + match + "!");
17         }
18         matcher.appendTail(result);
19
20         System.out.println(result.toString());
21     }
22 }
```

Здесь паттерн довольно простой – `[a-z][A-Z]`. В отличие от предыдущих заданий, в этом помимо нахождения нужной комбинации символов необходимо провести замену. Для этого я воспользовался методом `appendReplacement()`, который добавляет все символы до совпадения в переданную первым аргументом строку (`stringbuffer`, если быть точнее), а вторым аргументом принимает строку, которую он помещает вместо совпадения. Во второй аргумент я передал конкатенацию символов `!` и самого совпадения между ними. И последний штрих – вызов метода `appendTail()`, который добавляет в конец `result` оставшуюся часть строки (в моем случае это необязательно, так как в конце строки стоит совпадения и дальше ничего нет, но если бы после `yZ` был какой-либо еще текст, то без данного метода он бы не добавился в `result`).

Идем далее к четвертому заданию

```
1  import java.util.regex.*;
2
3  public class IPAddressValidator {
4
5      public static void main(String[] args) {
6          String[] testIPs = {
7              "192.168.1.1",
8              "255.255.255.255",
9              "0.0.0.0",
10             "256.100.50.0",
11             "192.168.1",
12             "192.168.01.1",
13             "asdasdasfff"
14         };
15
16         // String ipPattern = "";
17         String ipPattern = "^((25[0-5]|2[0-4]\\d|1\\d{2}|[1-9]?\\d)\\.){3}"
18             + "(25[0-5]|2[0-4]\\d|1\\d{2}|[1-9]?\\d)$";
19
20         Pattern pattern = null;
21         try {
22             pattern = Pattern.compile(ipPattern);
23         } catch (PatternSyntaxException e) {
24             System.out.println("Regex error: " + e.getDescription());
25             return;
26         } catch (Exception e) {
27             System.out.println("Unexpected error: " + e.getMessage());
28             return;
29         }
30
31         for (String ip : testIPs) {
32             Matcher matcher = pattern.matcher(ip);
33             if (matcher.matches()) {
34                 System.out.println(ip + " is a valid IP address.");
35             } else {
36                 System.out.println(ip + " is NOT a valid IP address.");
37             }
38         }
39     }
40 }
```

Все ранее упомянутые моменты я повторять не буду, поэтому переходим сразу к паттерну. Так как напрямую указать ограничения для чисел нельзя, пришлось идти не некоторые исключения, разобрав 4 возможных варианта, соединенных символом |.

Первый 25[0-5] охватывает числа от 250 до 255

Второй 2[0-4]\\d – числа от 200 до 249

Третий 1\\d{2} – от 100 до 199

Четвертый [1-9]?\\d – от 0 до 99

В конце ставится точка \\. и данная комбинация повторяется 3 раза, затем то же самое, только 1 раз и без точки на конце.

И в заключение – задание 5 на поиск слов, начинающихся с заданной буквы

```
1  import java.util.regex.*;
2
3  public class WordFinder {
4
5      public static void main(String[] args) {
6          String text = "Apple apricot banana Avocado berry Almond";
7          char targetLetter = 'A';
8
9          String patternString = "\\b" + targetLetter + "\\w*\\b";
10
11         Pattern pattern = null;
12         try {
13             pattern = Pattern.compile(patternString);
14         } catch (PatternSyntaxException e) {
15             System.out.println("Syntax error: " + e.getDescription());
16             return;
17         } catch (Exception e) {
18             System.out.println("Unexpected error: " + e.getMessage());
19             return;
20         }
21
22         Matcher matcher = pattern.matcher(text);
23         boolean found = false;
24
25         while (matcher.find()) {
26             System.out.println(matcher.group());
27             found = true;
28         }
29
30         if (!found) {
31             System.out.println("No words starting with letter '" + targetLetter + "' found");
32         }
33     }
34 }
```

Тут вообще объяснять нечего - \\b ограничение для слова, потом сама буква + \\w* любое количество букв и цифр и после закрывающий спецсимвол \\b.
Далее все как обычно

Вывод

В ходе лабораторной работы мной были освоены строки и регулярные выражения в языке Java.

Ссылка на гит - <https://github.com/BestStarProgrammer/IT-P/tree/main/lab2>